運用 Vibe Coding 實作

Ameba Mini 語音控制 LED 系統

專題報告

課程／班級	物聯網概論與應用
學生姓名／學號	李倢瑜／615K0029C

摘要

本專題採用 Vibe Coding 作為開發方法，建置一套以電腦端語音辨識結合 Ameba Mini 開發板之 LED 控制系統。系統由電腦麥克風擷取使用者語音，透過 Python 執行語音辨識與指令解析，並以 USB Serial 將控制命令傳送至 Ameba Mini。系統支援繁體中文與英文語音輸入，可執行左右 LED 之開啟、關閉與各自閃爍三次等功能。

在軟體架構方面，本專題將語音辨識、指令解析及硬體控制分離設計。語音辨識結果先經由白名單式指令解析器判定，僅當文字內容符合預先定義之合法命令時，系統才會產生對應的 Serial 控制訊息，例如 `LED,BLUE,ON`、`LED,GREEN,OFF` 與 `LED,BLUE,BLINK,3`。對於非控制語句、辨識失敗或不完整指令，系統將回傳 `UNKNOWN` 並停止傳輸，以避免非預期語音造成 LED 狀態改變。

硬體端採用 AMB82-MINI（RTL8735BDM）開發板，已完成程式燒錄與 USB Serial 通訊架構建置。板載藍色 LED 對應 PF9／Arduino pin 23，綠色 LED 對應 PE6／Arduino pin 24；目前程式設定以 `HIGH` 控制亮燈、以 `LOW` 控制熄燈。開發板於完成命令執行後，會回傳 `STATUS, BLUE=ON, GREEN=OFF` 等狀態訊息，使電腦端可根據硬體回傳資料呈現 LED 狀態，而非僅以命令是否送出作為成功依據。

此外，本系統針對通訊異常設計錯誤處理機制。當 USB Serial 連線中斷、COM 埠遭其他程式占用，或系統未收到有效狀態回覆時，程式會顯示通訊失敗訊息，並將 LED 實際狀態標示為未知。目前已完成中英文開關燈、閃爍三次、非控制語句保護及通訊中斷處理等功能，相關核心邏輯已通過 6 項單元測試。

本報告的重點

本報告旨在系統性說明本專題之整體設計理念、軟硬體架構、功能實作流程與驗證結果。內容涵蓋電腦端語音辨識、白名單式指令解析、USB Serial 通訊、Ameba Mini 韌體設計，以及藍色與綠色 LED 的開關與閃爍控制。此外，本報告亦整理 Vibe Coding 過程中的關鍵提示詞、AI 產生程式、測試結果與問題修正紀錄，以呈現生成式 AI 如何實際參與系統開發。

1 專題目標與需求分析

本專題的目標是透過自然語言描述需求，運用生成式 AI 協助產生、測試與修正程式，完成一套能以語音控制 Ameba Mini LED 亮滅的系統。系統不要求在 Ameba Mini 上執行語音辨識，因此本設計將辨識工作放在電腦端，以降低開發板運算與網路配置的複雜度。

1 1 功能需求

需求	設計回應	驗證方式
語音輸入與辨識	使用電腦麥克風與 Google Web Speech 辨識繁體中文。	以 --mic 模式說出指定指令，檢查辨識文字。
控制兩顆 LED	左邊開燈對應藍燈，右邊開燈對應綠燈；傳輸時轉成固定格式命令。	已完成板卡連接與程式燒錄；接下來以實體 LED 與板卡 STATUS 回傳驗證。
操作回饋	終端機顯示辨識結果、判定命令、通訊結果與狀態來源。	檢查輸出是否區分模擬回覆與開發板回覆。
非控制語句處理	未列入白名單的句子回傳 UNKNOWN，不傳送 Serial 資料。	以「今天天氣很好」等句子測試。
通訊失敗處理	捕捉 CommunicationError，顯示失敗與 LED 實際狀態未知。	模擬中斷；接板後以拔除 USB 測試。

1 2 選擇 USB Serial 的原因

本專題採用電腦端 Python 結合 USB Serial 的通訊架構，主要考量系統穩定性、實作複雜度與現場驗收的可控性。相較於 Wi-Fi 通訊，USB Serial 不需額外處理無線網路連線、IP 位址、路由器設定或網路環境變動等因素，能降低系統整合與除錯的複雜度，並使電腦端與 Ameba Mini 之間的命令傳輸流程更加明確。

在基本功能完成後，本專題進一步加入雙語語音命令與左右 LED 閃爍三次功能，使系統不僅能執行 LED 的開啟與關閉，也具備較多元的操作方式。

2 系統架構與資料流程

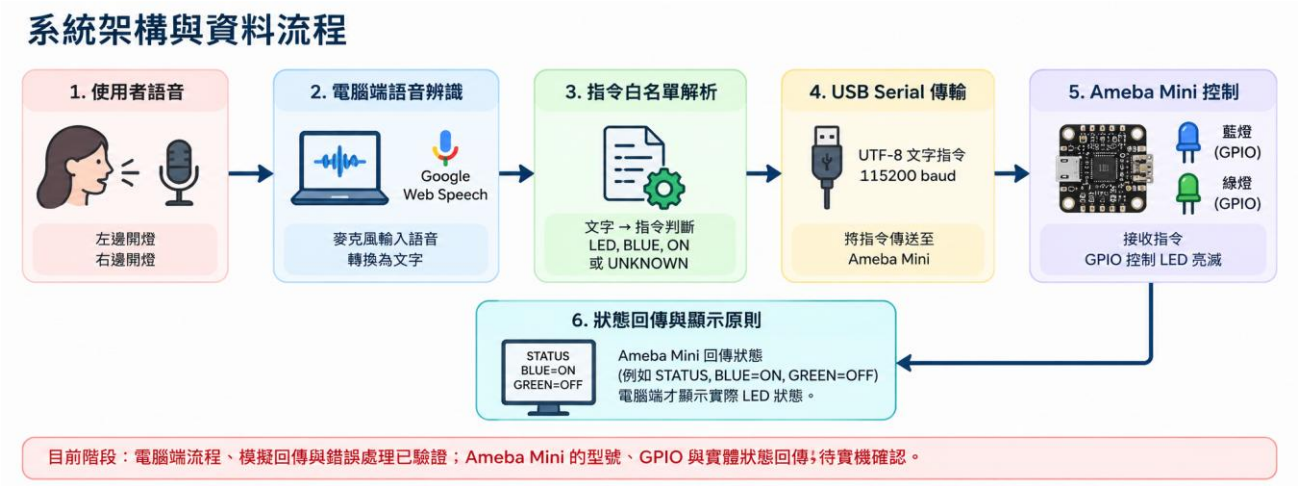


圖 1 系統架構與資料流程

2 1 狀態回傳的設計原則

本專題在系統回饋機制的設計上，電腦端不應僅因成功呼叫指令傳送函式，就直接判定「藍燈已亮」。於實體控制模式下，Ameba Mini 必須先依據接收的命令完成 GPIO 狀態設定，再透過 USB Serial 回傳格式正確的 STATUS 訊息，電腦端確認回覆內容後，才可將該結果視為 LED 的實際狀態。

此種雙向確認機制能區分「命令已送出」與「硬體已完成執行」兩種不同狀態，避免因 USB 連線中斷、Serial 讀取逾時、COM 埠異常，或開發板未正確執行命令時，系統仍顯示錯誤的成功訊息。因此，該設計可提升系統回饋資訊的可信度，並增強整體控制流程的可靠性與錯誤辨識能力。

3 電腦端程式設計

3 1 模組分工

檔案	角色	功能
main.py	系統入口與流程控制	顯示結果、建立控制器、處理語音與例外。
speech.py	麥克風語音辨識	使用 zh TW 與 Google Web Speech；失敗時不產生命令。
command.py	指令解析	採完全比對的白名單，不做模糊猜測。
serial_controller.py	通訊與狀態解析	支援 Mock 模式與日後的 Serial 模式。
test_command.py	核心單元測試	不需開發板即可測試解析、模擬狀態與中斷。

3 2 合法命令與白名單保護

指令解析器將辨識文字先移除空白與句尾標點，再與白名單完全比對。只有合法語句才能轉為控制命令；例如「左邊開燈」轉為 BLUE ON，「右邊開燈」轉為 GREEN ON。若輸入是「今天天氣很好」或「請幫我開所有燈」，系統回傳 UNKNOWN，主程式不會呼叫控制器，因此 LED 狀態維持不變。

辨識文字	內部命令	傳輸字串	行為
左邊開燈	BLUE ON	LED,BLUE,ON	開啟藍燈
左邊關燈	BLUE OFF	LED,BLUE,OFF	關閉藍燈
右邊開燈	GREEN ON	LED,GREEN,ON	開啟綠燈
右邊關燈	GREEN OFF	LED,GREEN,OFF	關閉綠燈
非控制語句	UNKNOWN	不傳送	LED 狀態不變
左邊閃爍三次	BLUE BLINK 3	LED,BLUE,BLINK,3	藍燈閃爍三次後熄滅
右邊閃爍三次	GREEN BLINK 3	LED,GREEN,BLINK,3	綠燈閃爍三次後熄滅

3 3 中英文指令與閃爍三次功能

功能	中文指令	英文指令	傳輸格式
左燈開啟	左邊開燈	turn on left light	LED,BLUE,ON
左燈關閉	左邊關燈	turn off left light	LED,BLUE,OFF
右燈開啟	右邊開燈	turn on right light	LED,GREEN,ON
右燈關閉	右邊關燈	turn off right light	LED,GREEN,OFF
左燈閃爍三次	左邊閃爍三次	blink left three times	LED,BLUE,BLINK,3
右燈閃爍三次	右邊閃爍三次	blink right three times	LED,GREEN,BLINK,3

3 4 操作與異常處理流程

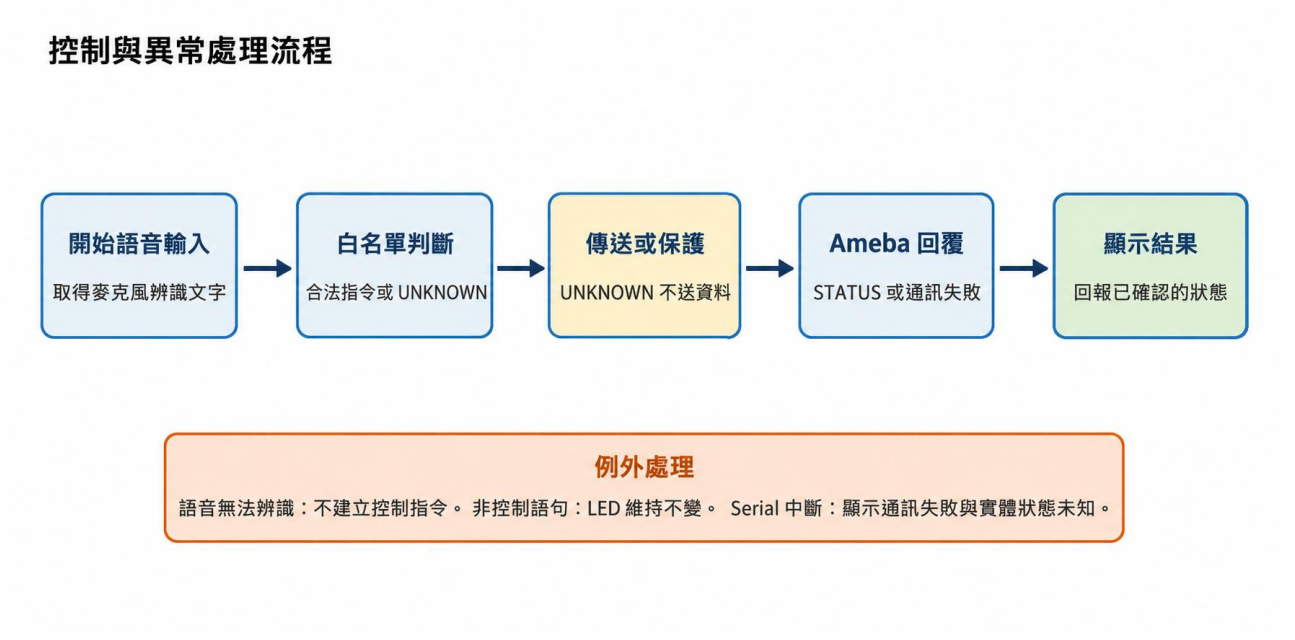


圖 2 控制與異常處理流程

4 通訊協定與狀態格式

電腦端與 Ameba Mini 使用 UTF 8 編碼的單行文字協定。每個控制命令尾端加上換行字元，Ameba Mini 讀取後控制 GPIO，並回傳包含兩顆 LED 狀態的 STATUS 訊息。此格式易於用序列監控視窗觀察，也利於日後除錯。

方向	格式	用途
電腦 → Ameba	LED,BLUE,ON	要求開啟藍色 LED
電腦 → Ameba	LED,GREEN,ON	要求開啟綠色 LED
Ameba → 電腦	STATUS,BLUE=ON,GREEN=OFF	告知控制後的兩顆 LED 狀態
錯誤情況	逾時、空字串或非 STATUS 開頭	電腦端顯示通訊失敗，不宣稱控制成功
電腦 → Ameba	LED,BLUE,BLINK,3	藍燈閃爍三次，結束後為關閉狀態
電腦 → Ameba	LED,GREEN,BLINK,3	綠燈閃爍三次，結束後為關閉狀態
Ameba → 電腦	READY,AMB82-MINI	開發板啟動後回報已就緒

實體模式的確認條件

本專題只有收到包含 BLUE 與 GREEN 完整狀態的 STATUS 訊息，程式才會顯示「Ameba 開發板實際回覆」。Mock 模式則明確標示為「非實體 LED 驗證」。

5 操作說明與執行環境

5 1 軟體環境

項目	目前設定
作業系統	Windows
Python	Python 3.12
套件	pyserial、SpeechRecognition、PyAudio
語音辨識服務	Google Web Speech，語言設定為 zh TW，需要網路
電腦與板卡通訊	USB Serial，預設 115200 baud

5 2 無開發板時的操作

即使已連接 Ameba Mini，仍可使用文字模擬模式先檢查指令解析與錯誤處理。這個模式不會控制實體 LED，但能驗證電腦端從輸入文字、命令判斷到模擬回覆的完整流程。

```
py -3.12 -m pip install -r requirements.txt
py -3.12 -m unittest -v
py -3.12 main.py
py -3.12 main.py --simulate-disconnect
py -3.12 main.py --mic
```

5 3 預期操作畫面

```
辨識結果：左邊開燈
判定：BLUE_ON；準備傳送：LED,BLUE,ON
指令已獲確認。
模擬 Ameba 回覆（非實體 LED 驗證）：藍燈=亮，綠燈=滅
```

5 4 接上開發板後的操作

開發板已成功燒錄後，確認裝置管理員中的 COM 埠與板卡韌體，再改用 Serial 模式，例如：

```
py -3.12 main.py --mode serial --port COM4 --baudrate 115200
```

中文語音使用預設 --language zh-TW；英文語音使用 --language en-US，例如：py -3.12 main.py --mode serial --port COM4 --mic --language en-US

本專案目前使用 COM4；若 Windows 裝置管理員中的編號改變，必須以實際顯示結果取代。

6 Ameba Mini 硬體整合計畫

本專題目前已完成初步開發環境建置與硬體連線測試，並成功將程式燒錄至 Ameba Mini 開發板中。此階段主要目的為確認後續系統開發所需的基礎環境是否能夠正常運作，包括電腦端開發工具設定、USB 連線狀態、開發板辨識情形，以及程式編譯與燒錄流程等。經實際測試後，程式已可正常完成編譯並成功寫入開發板，代表目前軟硬體之間的基本連線與開發流程已建立完成，可作為後續功能實作與系統整合的基礎。

完成上述基礎環境建置後，下一階段將進一步進行開發板實機功能確認，包含板卡型號與硬體規格確認、GPIO 腳位配置、LED 輸出控制，以及程式執行後的實際反應測試。由於不同版本或型號的開發板可能在 GPIO 編號、內建 LED 腳位及硬體配置上存在差異，因此本專題在實作過程中，將以實際使用之 Ameba Mini 開發板標示、官方技術文件與官方範例程式作為主要參考依據，並透過實際測試逐步驗證各項功能。

透過此階段的實機確認，可確保開發板能夠正確接收程式指令並控制對應硬體元件，同時降低後續進行語音辨識、指令傳輸及 LED 控制整合時，因硬體腳位設定或開發環境問題所造成的錯誤。待 GPIO 與 LED 基本控制功能確認完成後，即可進一步進入系統主要功能的開發與整合階段。

6 1 硬體整合順序

1. 先單獨燒錄藍燈與綠燈的亮滅程式，確認腳位與 active high 或 active low 邏輯。
2. 再讓 Ameba Mini 從 Serial 讀取一行 LED,BLUE,ON，確認單一文字指令可控制 LED。
3. 加入 STATUS 回傳，確認電腦端能解析兩顆 LED 的狀態。
4. 最後才切換到麥克風輸入，進行完整語音驗收。

7 Vibe Coding AI 協作紀錄

本專題在系統開發過程中導入生成式 AI 作為程式設計與問題分析的輔助工具，但並非直接採用 AI 所產生的程式碼，而是透過「需求描述、程式產生、實際測試、問題發現、程式修正與再次驗證」的反覆流程進行。實作時，先依據專題需求向 AI 說明預期功能、硬體環境與控制方

式，再將 AI 提供的程式架構或建議實際應用於 Ameba Mini 開發板，透過編譯、燒錄與實機操作確認系統是否能正常運作。

當測試結果與預期不符時，則依據錯誤訊息、GPIO 腳位狀態、LED 反應及通訊結果進行人工判斷，並將實際測試情形回饋給 AI，作為後續修正與調整的依據。因此，AI 在本專題中主要扮演程式撰寫、除錯建議與技術說明的輔助角色，而最終功能是否正確仍需透過人工實機驗證。以下將整理數個具代表性的 AI 協作紀錄，以說明本專題的實際開發與驗證流程。

7 1 協作紀錄一：建立可在無板卡環境測試的版本

項目	內容
需求提示詞	建立 Python 專案，將「左邊開燈」控制藍燈、「右邊開燈」控制綠燈；尚未有 Ameba 時仍可測試，非控制語句不可改變 LED。
AI 產生內容	command.py、serial_controller.py、main.py 與 test_command.py。
人工確認	將控制器區分為 Mock 模式與 Serial 模式，且在畫面上標示模擬狀態不是實體 LED 證據。
測試結果	以 unittest 執行 5 項核心測試，全部通過。

7 2 協作紀錄二：處理非控制語句

項目	內容
需求提示詞	當使用者說「今天天氣很好」等非 LED 控制指令時，不應改變 LED 狀態，請修改 Python 程式。
AI 建議	加入指令白名單與 UNKNOWN 判斷；未知文字不寫入 Serial。
人工驗證	測試程式確認「今天天氣很好」與「請幫我開所有燈」皆解析為 UNKNOWN。
結果	避免語音辨識誤判或一般對話造成意外控制。

7 3 協作紀錄三：處理 Windows 編碼問題

項目	內容
實際問題	執行 pip install -r requirements.txt 時出現 CP950 無法解碼 UTF 8 中文註解的 UnicodeDecodeError。

原因分析	Windows 環境以 CP950 讀取 requirements.txt，而檔案內中文註解使用 UTF 8。
修正提示詞	保留套件名稱，將 requirements.txt 的註解改成 ASCII，避免終端機編碼造成 pip 解碼失敗。
修正與驗證	將註解改為英文後重新安裝成功，並確認 pyserial、SpeechRecognition、PyAudio 可正常匯入。

AI 協作的驗證原則

AI 提供程式與修正方向後，仍以實際執行指令、單元測試、程式輸出與日後實體板卡反應作為是否採用的依據。

8 已完成的測試與結果

下表為截至 2026 年 10 月 1 日在無開發板環境完成的測試。測試對象是電腦端程式與 Mock 控制器，因此可證明程式流程與異常處理，但不能代替 LED 的實體亮滅驗證。

日期	測試模式	輸入或項目	結果	結論
2026.10.01	unittest mock	四種合法指令	正確解析；5 個單元測試通過	**成功**
2026.10.01	unittest mock	今天天氣很好等非控制句	回傳 UNKNOWN，不傳送命令	**成功**
2026.10.01	unittest mock	模擬通訊中斷	拋出 CommunicationError，主流程可顯示失敗	**成功**
2026.10.01	安裝環境	pip install -r requirements.txt	CP950 解碼失敗；改 ASCII 註解後安裝成功	**已修正**
2026.10.01	硬體燒錄	Ameba Mini 連接與程式燒錄	已連接板卡並成功燒錄程式；LED 實際反應尚未記錄	**已完成**
2026.10.01	unittest	中英文開關與閃爍三次	6 項單元測試通過	**成功**

8 1 已知限制

1. 尚未用本人麥克風完成每個指令 5 次的真實語音辨識紀錄，因此仍需在安靜與一般環境各測試一次。
2. Mock 控制器回覆只用於驗證軟體流程，不能證明藍燈或綠燈已在板上亮起。
3. Google Web Speech 需要網路；網路失敗時程式應顯示語音辨識失敗，且不產生控制命令。

9 驗收測試紀錄表

本節依照作業驗收規則設計。取得板卡後，應以終端機輸出、Ameba STATUS 回覆與 LED 實際觀察填寫。每一次「成功」必須同時符合：語音辨識文字正確、命令正確、板卡回傳狀態有效、LED 實際反應符合預期。

項目	次數	辨識文字	命令	結果
左邊開燈	1	左邊開燈	LED,BLUE,ON	已成功
左邊開燈	2	左邊開燈	LED,BLUE,ON	已成功
左邊開燈	3	左邊開燈	LED,BLUE,ON	已成功
左邊開燈	4	左邊開燈	LED,BLUE,ON	已成功
左邊開燈	5	左邊開燈	LED,BLUE,ON	已成功
右邊開燈	1	右邊開燈	LED,GREEN,ON	已成功
右邊開燈	2	右邊開燈	LED,GREEN,ON	已成功
右邊開燈	3	右邊開燈	LED,GREEN,ON	已成功
右邊開燈	4	右邊開燈	LED,GREEN,ON	已成功
右邊開燈	5	右邊開燈	LED,GREEN,ON	已成功
非控制語句	1	今天天氣很好	不傳送	已成功
通訊中斷	1	沒說話	LED,BLUE,ON	已成功

10 GitHub 與示範影片規劃

10 1 GitHub 專案內容

資料或檔案	用途
main.py 、 speech.py 、 command.py 、 serial_controller.py	完整電腦端原始程式。
requirements.txt	可重現的 Python 套件設定。
test_command.py	不需開發板的核心單元測試。
ameba 目錄中的韌體程式	接收 Serial 指令、控制 GPIO、回傳 STATUS。
README.md	安裝、執行、通訊協定、硬體設定與測試步驟。
docs 或 screenshots	系統架構、測試紀錄與操作畫面。

示範影片已上傳至 YouTube，請點擊觀看：**Ameba Mini 語音控制 LED 實作影片**

GitHub 專案連結：目前尚未建立。完成版本整理後，請在此處點選連結。

11 學習心得

透過本專題的實作，我對 Vibe Coding 的開發方式有了更具體的理解。Vibe Coding 並不是將完整需求一次交由 AI 處理後直接取得最終成品，而是將原本較大的系統需求拆解成多個可以分別驗證的小步驟，再逐步完成與整合。例如這次先從語音轉文字功能開始，接著依序完成指令白名單、模擬控制器、Serial 通訊協定與錯誤處理機制。透過這種分階段驗證的方式，即使在尚未取得實體開發板的情況下，也能先確認大部分的軟體流程是否正確。當硬體準備完成後，便可以將測試重點集中在 GPIO 腳位控制、程式燒錄與 Serial 實際收發等硬體相關項目，降低同時處理多項問題所造成的除錯困難。

而在系統驗證的過程中，我也進一步理解「程式執行成功」與「整體系統運作成功」之間的差異。以 Mock 模式為例，程式可以模擬回傳「藍燈已開啟」的狀態，但這僅代表軟體邏輯執行正確，並不能證明實體開發板上的 LED 已經實際亮起。因此，本專題在實體模式中加入狀態回傳機制，要求系統必須收到 Ameba 開發板回傳的 STATUS 訊息後，才將 LED 狀態顯示為已成功控制。透過此設計，可避免系統僅根據電腦端指令推測硬體狀態，使操作結果更加可靠，也能提供較明確的使用者回饋。

此外在實作過程中遇到的 CP950 與 UTF-8 編碼問題，也讓我體會到 AI 所產生的程式仍必須回到實際開發環境中進行驗證。AI 可以協助分析錯誤訊息、提出可能原因與修正方向，但最終是否能正常執行，仍需透過套件安裝、環境設定、重新執行與實機測試確認。目前本專題已完成 Ameba Mini 開發板連接與程式燒錄，並完成示範影片、GitHub 專案整理與相關文件。